Faculty of Engineering and Technology

Electrical and Computer Engineering Department

Computer Networks – ENCS3320

Project #1

Socket Programming

Prepared by:

Name: Mustafa Altaweel

UID: 1221017

Abstract

This project aims to help students to gain a better understanding of socket programming and computer networking, by providing practical experience with fundamental key network commands, and focusing on understanding, implementation and creation of two important protocols; Transmission control protocol (TCP), and User datagram protocol (UDP) sockets.

Contents

Table of Figures

Theory:

Socket programming basics:

Socket programming is the foundation of network communication between devices, enabling data exchange over a network. Sockets provide an endpoint for sending or receiving data and support two key communication protocols: **Transmission Control Protocol (TCP)** and **User Datagram Protocol (UDP)**.

TCP (Transmission Control Protocol):

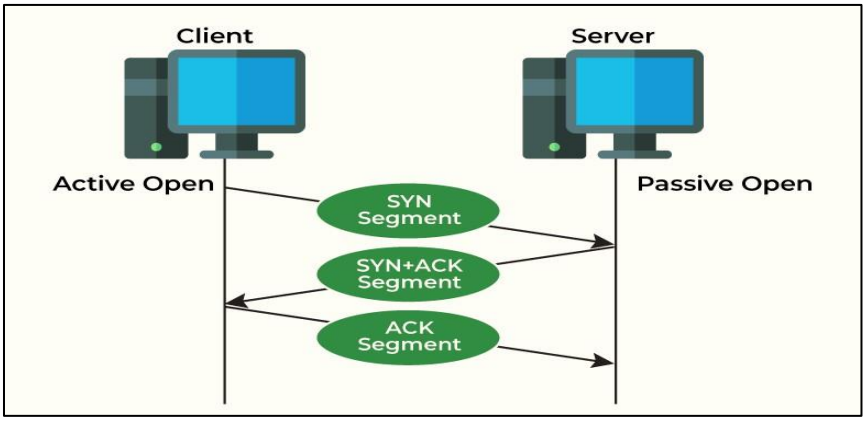


Figure 1 TCP Diagram

TCP is a connection-oriented protocol, which means it establishes a reliable communication channel between the client and server before transmitting data. This ensures data integrity, as packets are delivered in the correct order and without loss. TCP is commonly used in applications like web browsing and email.

Key Features:

1- Reliable and error-checked data delivery.
2- Stream-oriented communication.
3- Slower than UDP due to overhead for reliability.

UDP (User Datagram Protocol):

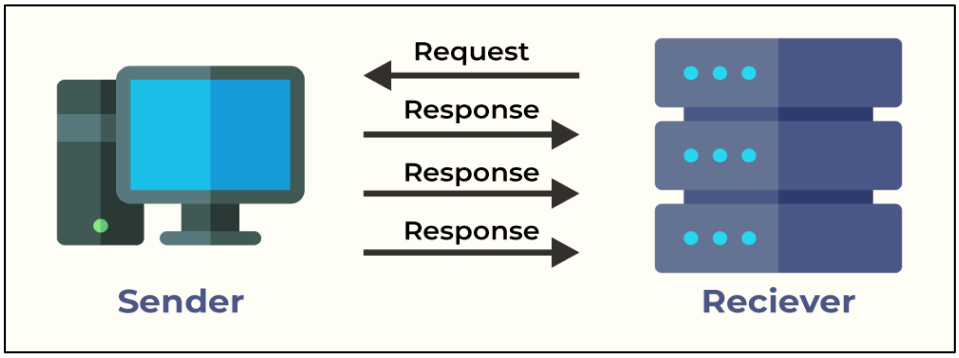


Figure 2 UDP Diagram.

UDP is a connectionless protocol, which means data is sent without establishing a connection. It is faster than TCP but does not guarantee delivery or order of packets. UDP is commonly used in applications requiring low latency, such as live video streaming and online gaming.

Key Features:

1- Faster but less reliable.
2- Packet-oriented communication.
3- Suitable for time-sensitive applications.

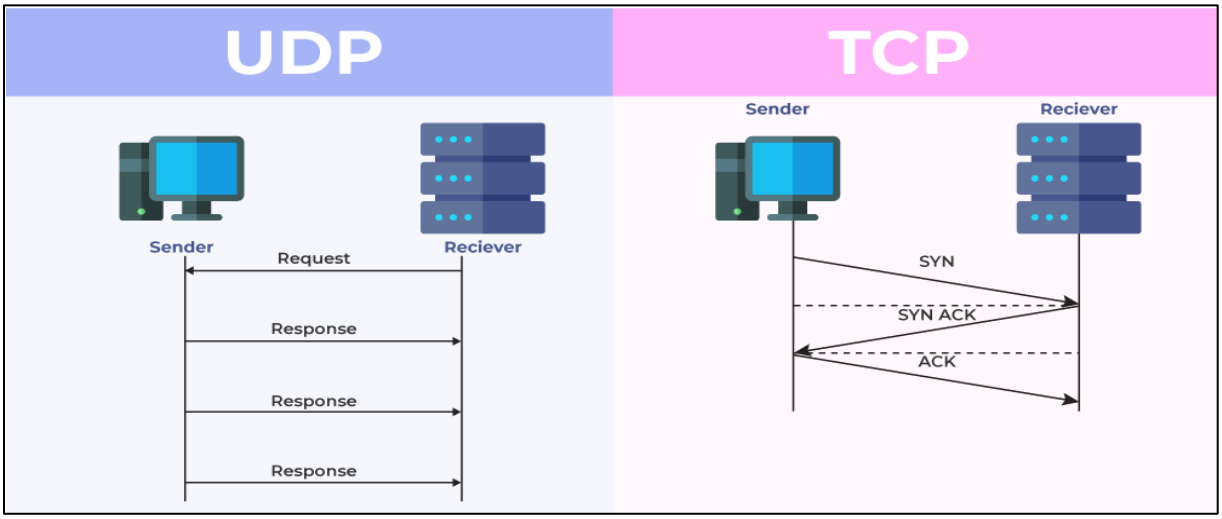


Figure 3 Comparison diagram between TCP and UDP.

Methodology and Tools:

Programming Languages:

To implement socket programming, various programming languages like **Python** and **C** are commonly used.

Python:

Python is widely favored for its simplicity and extensive library support. The built-in socket library makes it easy to create TCP and UDP sockets with minimal code, we used python because it has a **High-level abstraction**, and a **Fast development cycle**

UDP Server Client Game:

We are required to make a game that contains server and client side using UDP (User Datagram Protocol), we decided to make the program using python language due to its simplicity and functionality.

a. Server Side:

In server side, we started our code by some configurations, server IP, port number, and buffer size.

```
import socket
import random
import time

PORT = 5689                                 # The port number that will be used according to the project specifications.
SERVER = socket.gethostbyname(socket.gethostname())# The IP of the host that is using the server.
ADDR = (SERVER, PORT)                       # The data that will be written on the packet to route it correctly(IP, and Port used).
FORMAT = "utf-8"                            # Encoding format since the messages will be sent in binary format.
DIS_MESSAGE = "."                           # The sent message in case of disconnecting from the server.
BUFFER_SIZE = 1024                          # Maximum message size.
```

Figure 4 server information

We made a socket object with **datagram** type to confirm that we are using UDP, which will handle the connections with any client willing to join.

```
server_connect = socket.socket(socket.AF_INET, socket.SOCK_DGRAM) # Creates a new socket object wi
server_connect.bind(ADDR)                                  # Associates the socket with the specified addr
```

Figure 5 socket object initiate

After server initialization, we made the game functions, and the first one that will handle players and take their IP's and names, and notify them about game or other clients updates, which called *handle_client()*.

```
            time.sleep(3)
            print(f"[START] The game started!")
            start_game() # Start game
        continue

    username = clients[address]
    print(f"[{username} - {address}] {msg}")

    if msg == DIS_MESSAGE:                                  # If the client send the disconnection messege
        print(f"[DISCONNECTION] {username} at {address} disconnected.")
        del clients[address]
        del scores[address]
        connections -= 1
        continue

    server_msg = "Message received".encode(FORMAT)
    server_connect.sendto(server_msg, address)
```

Figure 6 Client handling function

Other function we made is called *start_game()*, which purpose is to randomly pick number of rounds and questions each round, and uses another function, *send_question_to_all_clients()*, to send the questions updates to all players, and when number of rounds is exceeded, we call *end_game()* function to notify all players that the game has ended.

```python
def start_game():
    for round_num in range(1, 4):   # Randomly choose number of rounds
        print(f"[ROUND {round_num}] Starting round {round_num}...")
        for client in clients:
            server_connect.sendto(f"[ROUND {round_num}] Get ready!".encode(FORMAT), client) # Notify all clients the round number

        num_questions = random.randint(2, 4)    # Randomly choose the number of questions for the current round
        for _ in range(num_questions):
            send_question_to_all_clients()      # Send the Question to all clients

        print(f"[ROUND {round_num}] Round {round_num} completed!")

    end_game() # End the game after rounds finishes
```

```python
def send_question_to_all_clients():

def end_game():
    print("[GAME OVER] Ending the game.")
    for client in clients:
        server_connect.sendto("game over".encode(FORMAT), client)
    print("[FINAL SCORES]")
    for client, score in scores.items():
```

Figure 7 Send question to all players function

Figure 8 End game function

Helper methods that are used for server side are *collect_answers()* that collects all clients answers and see if they answered correctly. Also, *send_scoreboard()* that broadcasts each question the players scoreboard to all players.

```python
def collect_answers(question):
    start_time = time.time()
    received_answers = {}

    while time.time() - start_time < 15: # Set timer of 15s for each round
        try:
            server_connect.settimeout(15 - (time.time() - start_time))
            answer_msg, client = server_connect.recvfrom(BUFFER_SIZE)   # Collect answers from clients
            answer_msg = answer_msg.decode(FORMAT)

            if client not in clients:
                continue

            if client not in received_answers:
                received_answers[client] = answer_msg
        except socket.timeout:
            break

    for client, answer in received_answers.items():
        if answer.lower() == question["answer"].lower(): # If answer is correct
            scores[client] += 1
            print(f"[CORRECT ANSWER] {clients[client]} answered correctly! New score: {scores[client]}")
            server_connect.sendto(f"You answered correctly! Your score now is: {scores[client]}".encode(FORMAT), client)
        else:                                           # If answer is wrong
            print(f"[WRONG ANSWER] {clients[client]} answered incorrectly.")
            server_connect.sendto(f"You answered wrong! Your score still: {scores[client]}".encode(FORMAT), client)
```

Figure 9 Collect answers function

```python
def send_scoreboard():
    scoreboard = "[SCOREBOARD]\n" + "\n".join([f"{clients[client]}: {scores[client]}" for client in clients])
    print(scoreboard)
    for client in clients:
        server_connect.sendto(scoreboard.encode(FORMAT), client)
```

Figure 10 Send scoreboard function

b. Client Side:

We treated the clients as the players of the game, which we need their IPs and names, so we started with some configurations as in server side. And made a socket object as in server side using **datagram** to successfully work on UDP protocol.

```python
import socket

PORT = 5689
SERVER = socket.gethostbyname(socket.gethostname())
ADDR = (SERVER, PORT)
FORMAT = "utf-8"
DIS_MESSAGE = "."
BUFFER_SIZE = 1024

client_connect = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

Figure 11 Client configurations

We started the connection using function called server_connection(), which is responsible for taking the client's name and IP, then check the game status and broadcast result to the player.

```python
def server_connection():
    client_name = input("Please enter your username: ")
    client_connect.sendto(client_name.encode(FORMAT), ADDR)
    print("Connected to the server.")

    while True:
        server_msg, _ = client_connect.recvfrom(BUFFER_SIZE)
        message = server_msg.decode(FORMAT)

        if message == "start":
            print("The game will start soon, be prepared!")
            break
        elif "joined the game" in message:
            print(f"[INFO]: {message}")
        elif message == "wait":
            print("Please wait for other players to join...")

    return True
```

Figure 12 Client connection function

We also the function, *send(msg)*, that is responsible for sending messages to the server, like name or question answers, and if there is a timeout the message won't be sent.

```python
def send(msg):
    try:
        if msg == DIS_MESSAGE:
            print("Disconnecting from server...")
            client_connect.sendto(DIS_MESSAGE.encode(FORMAT), ADDR)
            client_connect.close()
            return False
        else:
            message = msg.encode(FORMAT)
            client_connect.sendto(message, ADDR)
            server_msg, _ = client_connect.recvfrom(BUFFER_SIZE)
            print(f"[SERVER]: {server_msg.decode(FORMAT)}")
            return True
    except socket.timeout:
        print("[ERROR]: No response from server, disconnecting...")
        return False
```

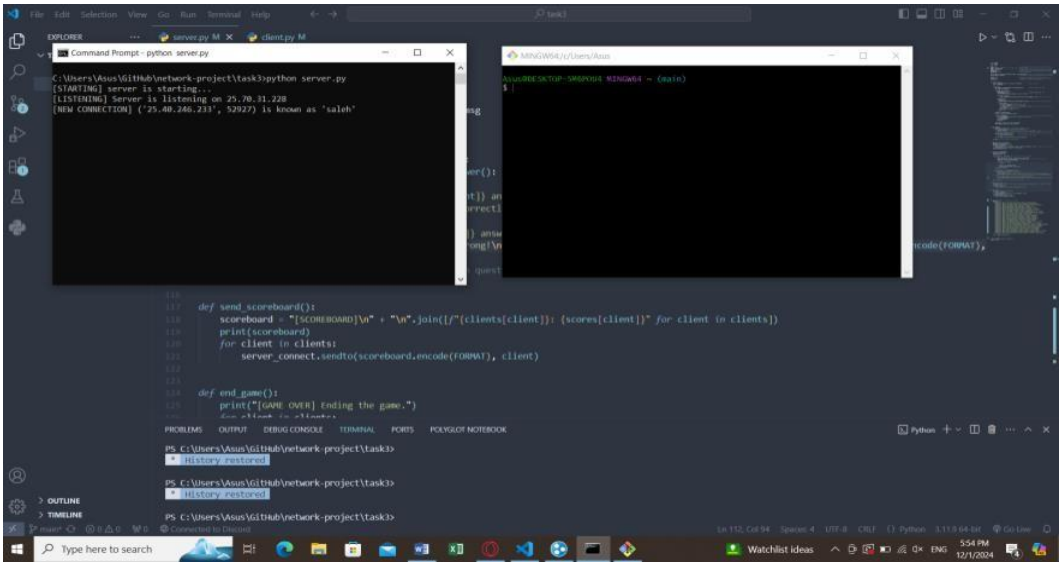
We reassembled all the functions above to one function called *main(),* which receive messages from the server and acts depending on the message info.

```python
def main():
    if server_connection():
        while True:
            try:
                server_msg, _ = client_connect.recvfrom(BUFFER_SIZE)
                message = server_msg.decode(FORMAT)

                if message == "game over":
                    print("The game has ended. Thank you for playing!")
                    break
                elif "[QUESTION]" in message:
                    print(f"{message}")
                    answer = input("Your answer: ")
                    if not send(answer):
                        break
                elif "[SCOREBOARD]" in message:
                    print(f"{message}")
                else:
                    print(f"[INFO]: {message}")
            except socket.timeout:
                print("[ERROR]: Server response timeout. Exiting game.")
                break

```

Figure 13 Main client function

C. Run Example:

Two Clients connected using **Hamachi** which allows you to setup a (VPN) Virtual Private Network that will allow you to share data between the computers in your network:



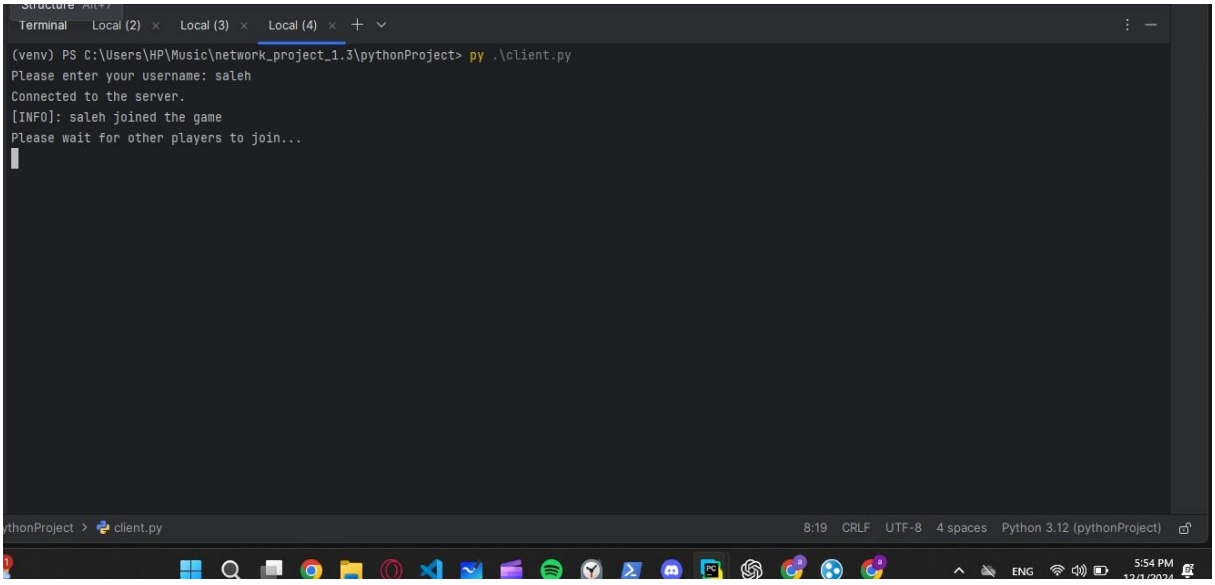
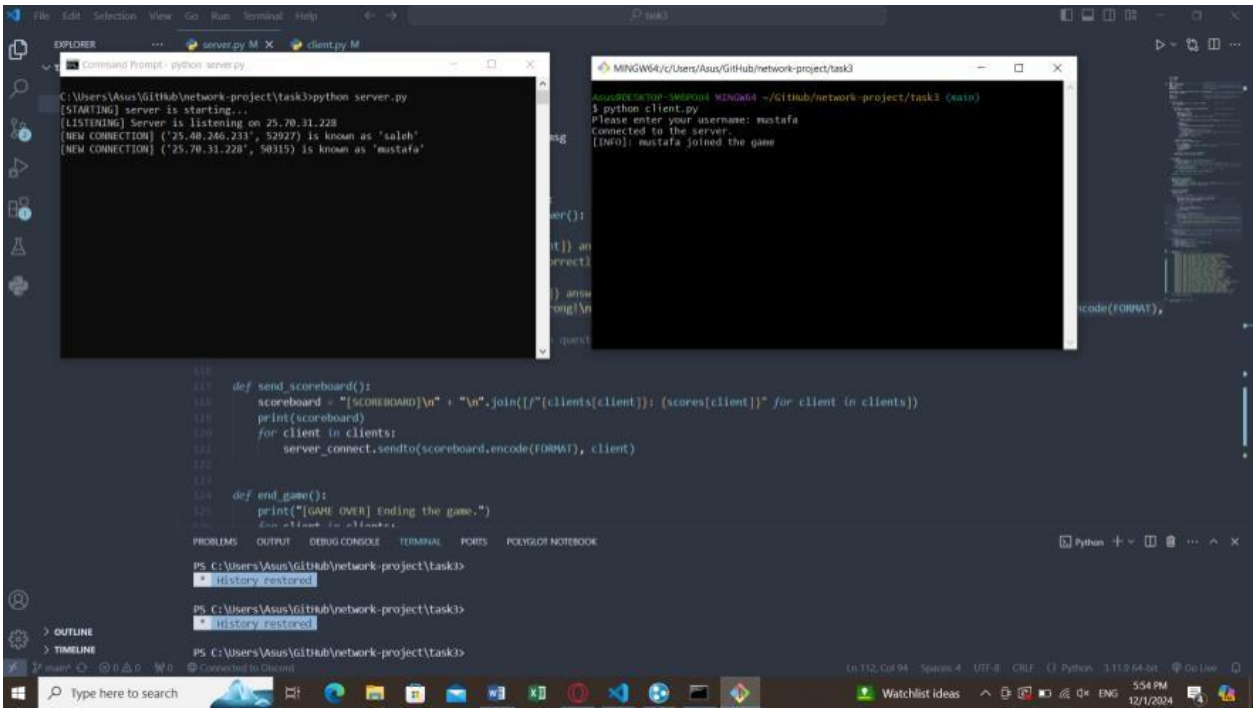


Figure 14 Example run (1).



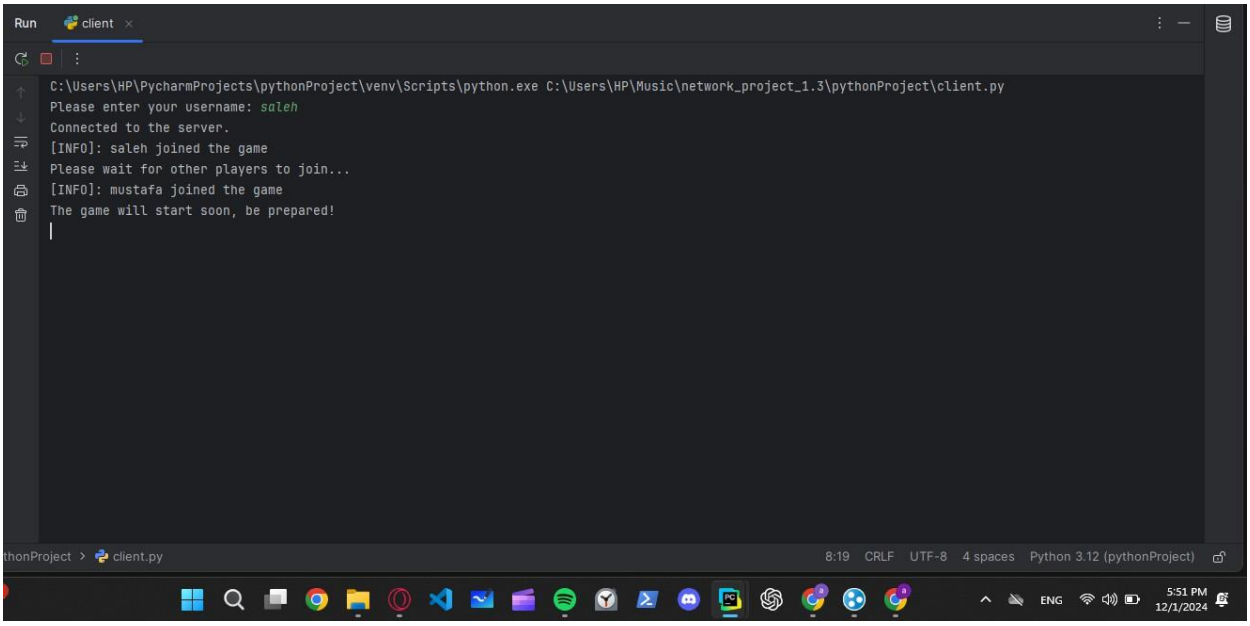
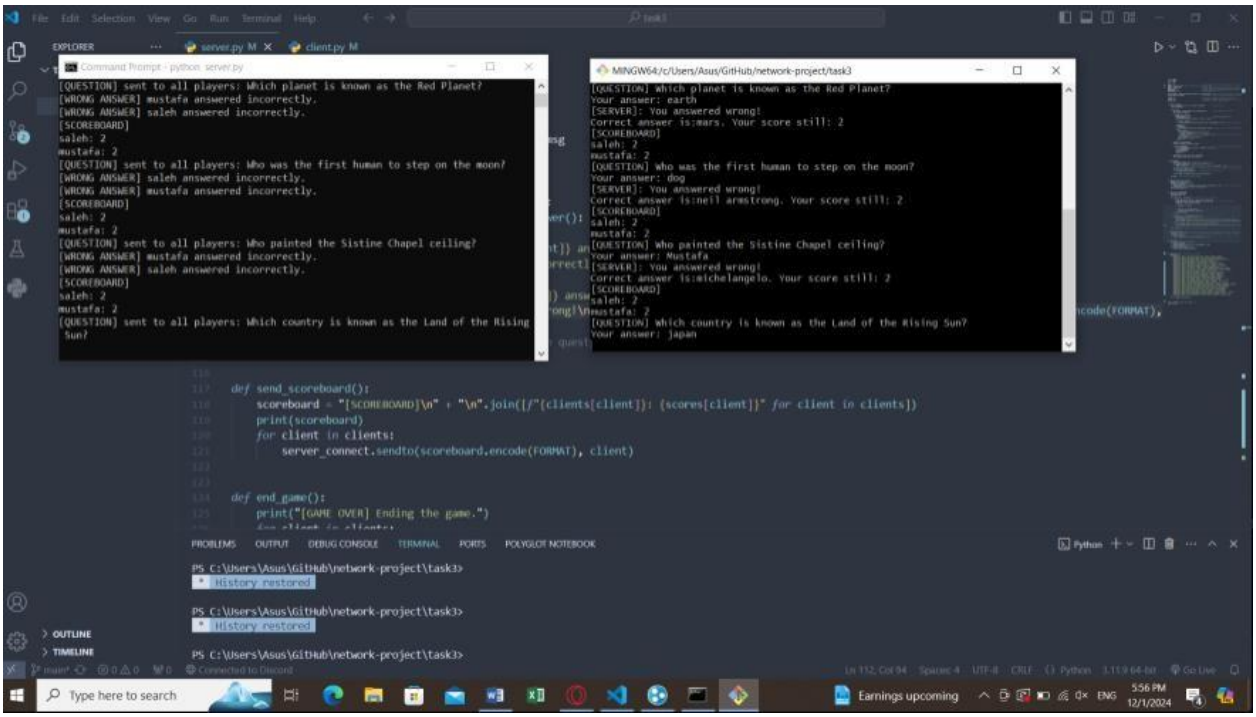


Figure 15 Example run (2).



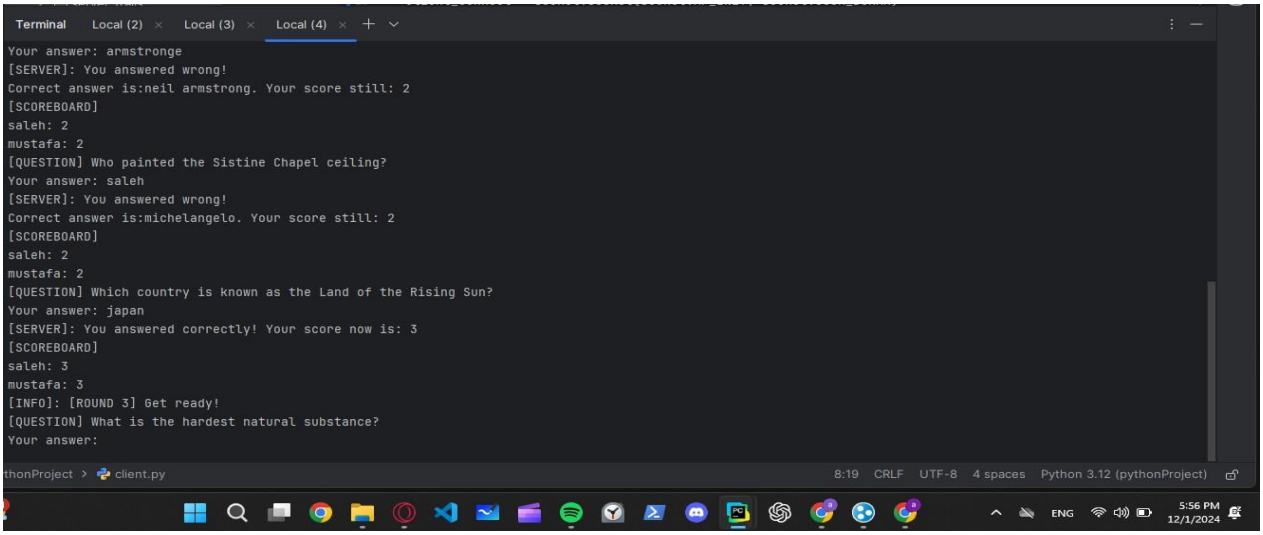
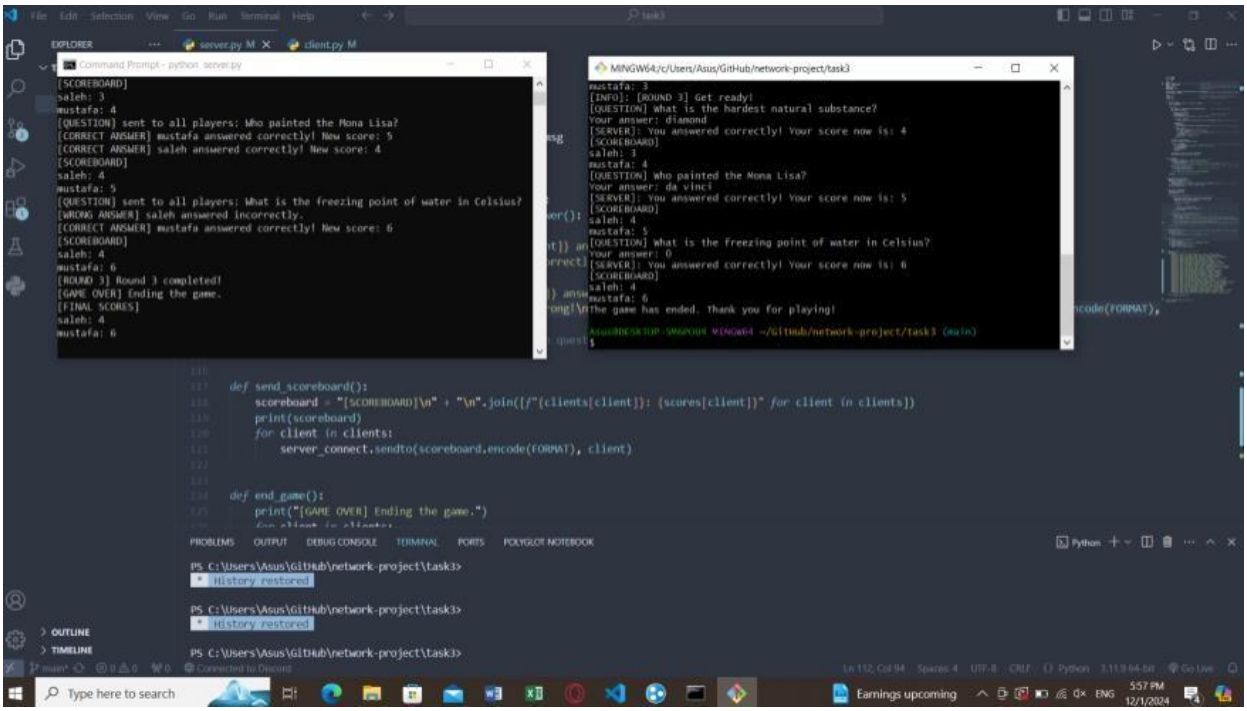


Figure 16 Example run (3).



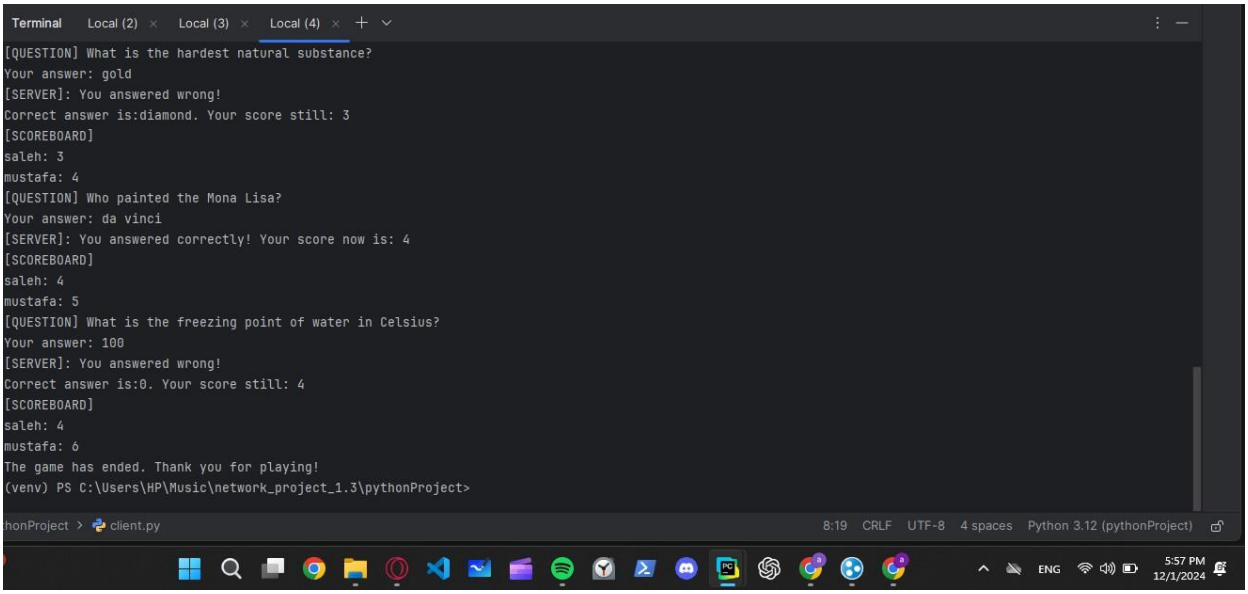


Figure 17 Example run (4).

Alternative Solutions, Issues, and Limitations:

Alternative Solutions:

C/C++ Programming:

In this project we used python for lots of reason mentioned in the theory part, we could have used C/C++, they offer lower-level control over system resources, which can be advantageous for performance-critical applications. By using the socket API in C, developers have direct access to memory management and can optimize network communication for speed. However, this comes at the cost of increased complexity and potential memory management errors. The reason why we didn't use C or C++ is that it is **much harder** and the **memory control** is **manual** which can introduce many **bugs** and **memory leaks**.

Issues, and Limitations:

Error Handling and Exception Management:

One of the challenges faced while using Python's socket library is handling network errors gracefully. Sockets in Python, especially when dealing with unreliable protocols like UDP, can experience packet loss or timeouts. While TCP guarantees packet delivery, UDP does not, which can result in data loss or the need for additional error-checking mechanisms.

References:

1- https://davisgitonga.dev/blog/request-response-cycle

2- https://www.geeksforgeeks.org/differences-between-tcp-and-udp/

3- Computer network a top down approach 8TH edition by Jim Kurose, Keith Ross